Theory of Automata

Project Final Report

Project Title: Regular Expression Matcher Using Java (Automatic Simulation)

Group Member:

Kashif Raza (IU05-0124-0303)

Areeba Fatima (IU05-0124-0342)

Course Instructor:

Miss Alisha Farman

Acknowledgement

I am deeply thankful to **Miss Alisha Farman** for their valuable guidance, continuous support, and encouragement throughout the development of this project. Their instructions and feedback played a vital role in enhancing my understanding of automata theory and its practical applications.

Introduction

Theory of Automata deals with abstract machines and formal languages. Regular Expressions are a key concept and are equivalent to Finite Automata. This project implements a Regular Expression Matcher using Java Swing, demonstrating how automata concepts are applied in real-world software.

Objectives

- Understand Regular Expressions
- Simulate Finite Automata behavior
- Validate strings using patterns
- Build a GUI-based automata application

Theoretical Background

Regular Expressions

A regular expression is a formal language used to describe a set of strings over a given alphabet. Regular expressions are equivalent to **Finite Automata**, meaning every regular expression can be converted into a DFA or NFA.

Finite Automata

Finite Automata consist of:

- A finite set of states
- An input alphabet
- Transition functions
- A start state
- One or more accepting states

When a string is processed, the automaton changes states according to the input symbols. If it ends in an accepting state, the string is accepted.

Relation to the Project

- The **regular expression** represents the automaton
- The **input string** represents the input symbols
- The Java regex engine simulates the automaton internally
- Acceptance or rejection is displayed as output

System Design

User Interface Design

The GUI contains:

- Text field to enter Regular Expression
- Text field to enter Input String
- Match button to perform validation
- Result label to display match status

Functional Flow

1. User enters a regular expression
2. User enters an input string
3. User clicks the **Match** button
4. System compiles the regular expression
5. Input string is matched against it
6. Result is displayed as:

    - ✅ Match Found
    - ❌ No Match
    - Invalid Regular Expression

Implementation Details

The project is implemented using Java Swing. The `Pattern` and `Matcher` classes are used to compile and match regular expressions.

Key functionalities include:

- Exception handling for invalid regular expressions
- Visual feedback using colors
- User-friendly interface

The program checks whether the **entire input string** matches the regular expression using `matches()` method.

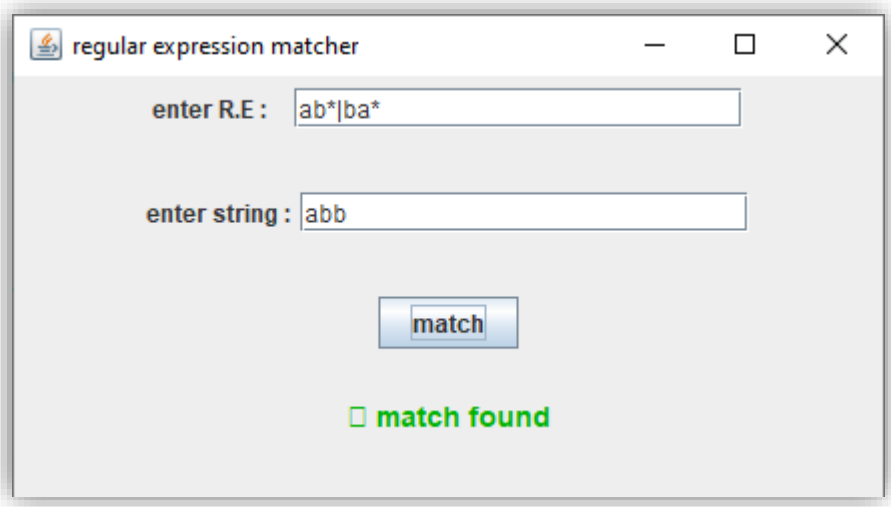
Regular Expression	Input String	Output
a*b	aaab	Match Found
0-9+	12345	Match Found
ab+	a	No Match
(	abc	Invalid Regular Expression

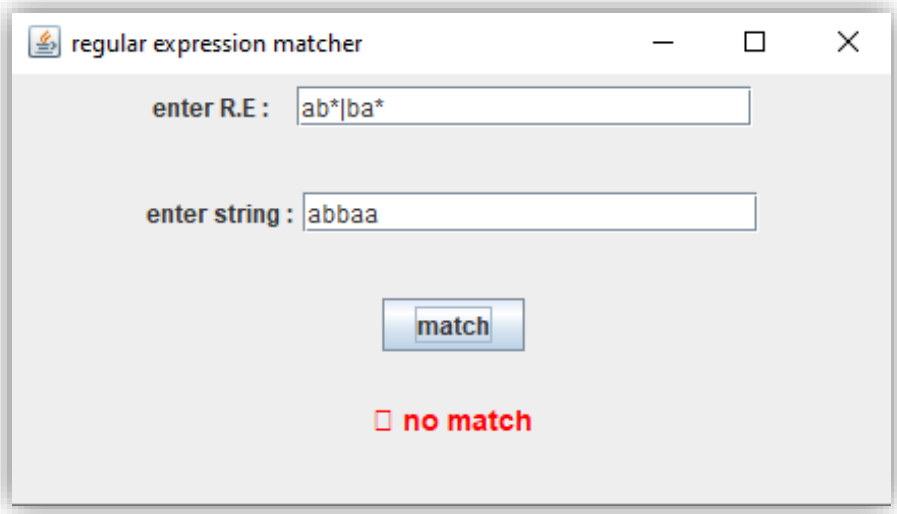
Java Source Code

```
package Kashif;

import javax.swing.*;
import  java.awt.*;
import java.awt.event.*;
import java.util.regex.*;

public class Kashif extends JFrame {

    private JTextField regf;
    private JTextField inputf;
    private JLabel rl;

    public Kashif() {
        setTitle("regular expression matcher");
        setSize(450, 250);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridLayout(4, 2));

        JPanel regp = new JPanel();
        regp.add(new JLabel("enter R.E :   "));
        regf = new JTextField(20);
        regp.add(regf);
        add(regp);

        JPanel ip = new JPanel();
        ip.add(new JLabel("enter string :"));
        inputf = new JTextField(20);
        ip.add(inputf);
        add(ip);

        JPanel bp = new JPanel();
        JButton mb = new JButton("match");
        bp.add(mb);
```

```
            add(bp);

            JPanel rp = new JPanel();
            rl = new JLabel("result will appear here...");
            rl.setFont(new Font("arial", Font.BOLD, 14));
            rp.add(rl);
            add(rp);

            mb.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e) {
                    performmatching();
                }
            });

            setLocationRelativeTo(null);
            setVisible(true);
        }

        private void performmatching() {
            try {
                String pt = regf.getText();
                String it = inputf.getText();

                Pattern p = Pattern.compile(pt);
                Matcher m = p.matcher(it);
                if (m.matches()) {
                    rl.setText("✔ match found");
                    rl.setForeground(Color.GREEN.darker());
                } else {
                    rl.setText("✖ no match");
                    rl.setForeground(Color.RED);
                }

    } catch (Exception ex) {
                rl.setText("invalid regular expression");
                rl.setForeground(Color.ORANGE);
            }
        }
        public static void main(String[] args) {
            new Kashif();
          }
        }
```

Output:


regular expression matcher
enter R.E :
ab*|ba*
enter string :
abb
match
match found


regular expression matcher
enter R.E :
ab*|ba*
enter string :
abbaa
match
no match

Limitations

- Does not visualize automaton states
- Depends on Java's built-in regex engine
- Matches only one string at a time
- No DFA/NFA construction shown

Applications

- Pattern matching in compilers
- Lexical analysis
- Input validation systems
- Search engines
- Text processing tools

Conclusion

This project successfully demonstrates the practical implementation of **Theory of Automata** concepts using Java. By building a Regular Expression Matcher, the project shows how abstract theoretical ideas such as finite automata and regular languages are applied in real-world programming. The project enhances conceptual clarity and provides a strong foundation for advanced topics like compiler design and formal language processing.

Future Enhancements

- Visual DFA/NFA generation
- Step-by-step transition display
- Multiple string testing
- File input support
- Enhanced UI design